

Modules

Modules

= a better way to organize our code

```
57 </div>
58
59 <script src="data/cart.js"></script>
60 <script src="data/products.js"></script>
61 <script src="scripts/amazon.js"></script>
62 </body>
63 </html>
```

1. Combine all the files together into 1 big file.
2. Run all the code.

```
/body>
html>
```

This can cause naming conflicts

```
1 const cart = [];
```

We can't use 'cart' in any other files.

Modules



cart.js

modules so a module basically contains a variable inside a file

Create a Module

1. Create a file
2. Don't load the file with `<script>`

Any variables we create inside the file, will be contained inside the file.

Get a Variable Out of a File

1. Add type="module" attribute
2. Export
3. Import

```
</div>  
<script src="data/products.js"></script>  
<script type="module" src="scripts/amazon.  
js"></script>
```

type="module" attribute

Let's this file get variables
out of other files.

this file so for example we want to
access the VAR variable cart

access the VAR variable cart outside of
this file to do that in front of

variable we're just going to type the
word export

```
export const cart = [];
```

```
import{ // name of the var we want
```

```
}
```

```
import{cart} from '../data/cart.js';
```

out of the scripts folder we're in the
scripts folder

scripts folder and we need to get out of
it and into the data folder

out of the current Cent folder that this
file is in we're going to type dot

so dot dot basically represents the
folder

folder outside of this current folder
which is this folder

we're going to type a forward slash to
go outside of the scripts folder

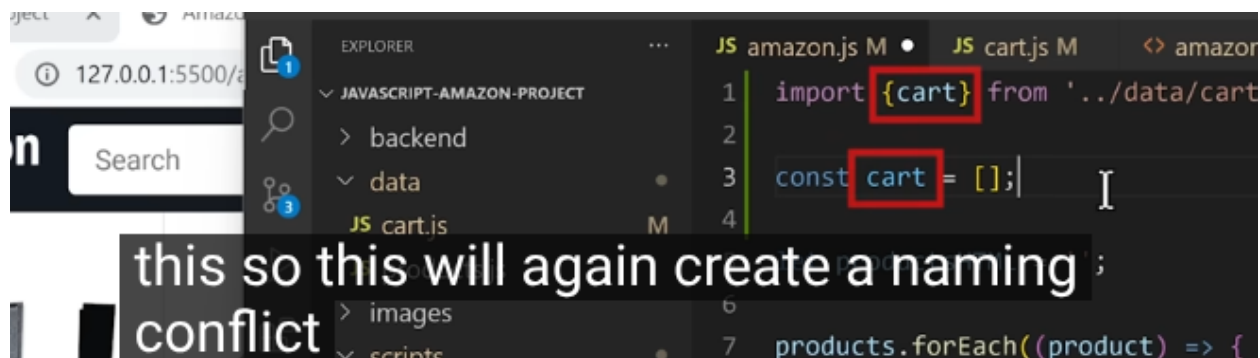
going to go into the data folder so here
we're going to type data

1. Put all imports at the top of the file.

2. We need to use Live Server

Benefits of Modules

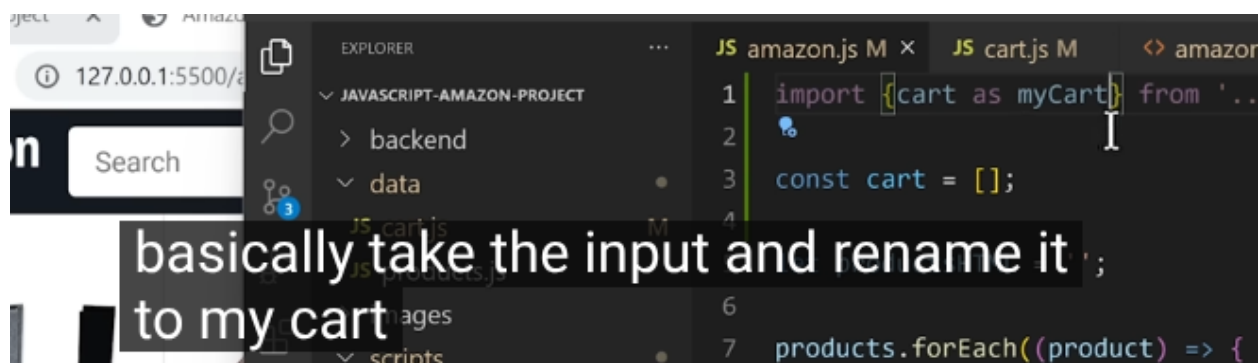
1. Helps us avoid naming conflicts



A screenshot of the Visual Studio Code editor. The Explorer sidebar on the left shows a project named 'JAVASCRIPT-AMAZON-PROJECT' with folders 'backend', 'data', 'images', and 'scripts'. The 'data' folder is expanded, showing 'cart.js'. The main editor area has two tabs: 'amazon.js M' and 'cart.js M'. The 'cart.js' file is active, showing the following code:

```
1 import {cart} from '../data/cart'
2
3 const cart = [];
```

The variable 'cart' in the import statement on line 1 and the variable 'cart' in the const declaration on line 3 are both highlighted with red boxes. A text overlay at the bottom of the screenshot reads: 'this so this will again create a naming conflict'.



A screenshot of the Visual Studio Code editor, similar to the previous one. The 'cart.js' file is active, showing the following code:

```
1 import {cart as myCart} from '../data/cart'
2
3 const cart = [];
```

The variable 'myCart' in the import statement on line 1 is highlighted with a blue box. A text overlay at the bottom of the screenshot reads: 'basically take the input and rename it to my cart'.

2. Don't have to worry about order of files

script tags we had to make sure that we loaded C.J first because we need the cart

Have to load cart.js first

```
<script src="data/cart.js"></script>
<script src="data/products.js"></script>
<script src="scripts/amazon.js"></script>
```

```
JS cart.js ×
1 const cart = [];
```

Modules = better way to organize our code

```
<script type="module" src="scripts/amazon.js"></script>
</body>
html>
```

Entry point

you'll notice that we're running a lot of code here when we click that button

we're running code that adds the product to the cart and we're also running code

we're running code that adds the product to the cart and we're also running code

that calculates the quantity and updates the page

the page so a best practice in programming is that

programming is that when we have a lot of code here that does different

it's better to split these up into smaller

smaller functions to make our code easier to

easier to read so here this part of the code takes the product ID and then adds

it to the cart so it makes sense to split this

let's create a function for this code so we'll type

```
//----- create another function for
readability--- //
function addToCart(){
  let matchingItem;

  cart.forEach((item)=>{
    if(productId === item.productId){
      matchingItem = item;
    }
  })

  if(matchingItem){
    matchingItem.quantity +=1;
  }
  else{
    cart.push ({
      productId: productId, // use productId
      instead of productName
      quantity : 1
    });
  }
};
```



```

document.querySelectorAll('.js-add-to-cart')
  .forEach((button)=>{
    button.addEventListener('click',()=>{
      const productId = button.dataset.productId;

      addToCart(); // call function // but not
                    working

      let cartQuantity = 0;
      cart.forEach((item)=>{
        cartQuantity += item.quantity;
      });

      document.querySelector('.js-cart-quantity')
        .innerHTML = `${cartQuantity}`;
      console.log(cart);
    });
  });
});

```

❗ Unchecked runtime.lastError: The message port closed before a r

2 ▶ Uncaught ReferenceError: productId is not defined
 at addToCart ([amazon.js:124:20](#))
 at HTMLButtonElement.<anonymous> ([amazon.js:137:7](#))

```

const productId = any
addToCart(productId); // call function //
now working

```

```

readability--- //
function addToCart(productId){

```

down and we're going to put the rest of our code into its own function as well

```
81 document.querySelectorAll('.js-add-to-cart')
82   .forEach((button) => {
83     button.addEventListener('click', () => {
84       const productId = button.dataset.productId;
85       addToCart(productId);
86
87       let cartQuantity = 0;
88
89       cart.forEach((item) => {
90         cartQuantity += item.quantity;
91       });
92
93       document.querySelector('.js-cart-quantity')
94         .innerHTML = cartQuantity;
95     });
96   });
```

```
function updateCartQuantity(){
  let cartQuantity = 0;
  cart.forEach((item)=>{
    cartQuantity += item.quantity;
  });

  document.querySelector('.js-cart-quantity')
    .innerHTML = `${cartQuantity}`;
  console.log(cart);
}

document.querySelectorAll('.js-add-to-cart')
  .forEach((button)=>{
    button.addEventListener('click', ()=>{
      const productId = button.dataset.productId;
      addToCart(productId);
      updateCartQuantity();
    });
  });
});
```

scroll up to the add to cart function so you'll notice that this code

manages our cart so it's actually a best practice to move this function into

cart. JS because cart. JS contains all the code that's related to the cart

Best practice

Group related code together into its own file.

```
AmazonProject > data > JS cart.js > addtoCart
1  export const cart = [];
2
3  export function addToCart(productId){
4
5      let matchingItem;
```

```
AmazonProject > scripts > JS amazon.js > ...
1
2  import {cart, addToCart } from '../data/cart.js';
3  import { products } from '../data/products.js';
4
```

```
import * as cartModule from '../data/cart.js';

cartModule.cart
cartModule.addToCart('id');
```

file Imports have another syntax Star as this Imports everything from a

file and groups it together inside this object and

cart. JS as ;
well so this function actually handles

updating the web page rather than
managing the cart so we're going to keep

this function inside this file for
now so